Trabalho Prático Integrado

Disciplina: Banco de Dados Avançado

Objetivo: Construção e entrega de uma solução computacional de ponta a ponta (*Full Stack*), contemplando automação de ambiente, banco de dados **SQL Server**, **API REST** e aplicação **Front-End**.

Visão Geral do Projeto

Cada grupo deverá desenvolver uma aplicação completa para resolução de um problema real ou fictício. O sistema deve ser totalmente funcional e contemplar desde a infraestrutura do banco de dados até a interface final do usuário, garantindo boas práticas de segurança, desempenho e manutenibilidade.

Estrutura do Projeto & Entregáveis

O trabalho deve ser entregue em um **único arquivo compactado (`.zip`)** nomeado com a identificação do grupo (ex.: `Grupo_01_TrabalhoFinal.zip`). O arquivo deve conter exatamente a seguinte estrutura:

```
\ Grupo_XX_TrabalhoFinal.zip
  ├──  Documentacao_Projeto.pdf
  ├──  Apresentacao_Projeto.pdf
  └── \ src_github/ (Pasta no formato de repositório GitHub)
        ├──  README.md
        ├──  setup_database.ps1 (Script de inicialização PowerShell)
        ├── \ database/ (Scripts T-SQL)
        ├── \ api/ (Código-fonte da API REST)
        └── \ frontend/ (Código-fonte da Aplicação Front-End)
```

Requisitos Técnicos

1. Banco de Dados (SQL Server) & Script PowerShell (.ps1)

- **Provedor:** Microsoft SQL Server.
- **Script PowerShell (.ps1):** Deve executar a criação automatizada do banco de dados, a estrutura de tabelas, o schema completo e a população com **dados de teste relevantes**.
- **Objetos Obrigatórios:** A modelagem deve obrigatoriamente aplicar os conceitos vistos em aula (*Stored Procedures*, *Functions*, *Views*, *Triggers*, tratamento de transações e estratégia de índices).

2. API REST

- Desenvolvida na linguagem/framework de preferência da equipe.
- Comunicação direta e funcional com o **SQL Server**.
- Implementação completa de endpoints para operações de **CRUD** (Create, Read, Update, Delete) das principais entidades.

3. Front-End

- Interface web ou mobile completa, amigável e funcional.
- Consumo da **API REST** desenvolvida.
- Permite ao usuário realizar interações completas (CRUD) com os dados.

4. Documentação Técnica (Documentacao_Projeto.pdf)

A documentação deve conter o detalhamento da arquitetura do sistema e **abordar obrigatoriamente todos os temas vistos no semestre**:

1. **Visão Geral do SQL Server e T-SQL Avançado:** Fundamentos aplicados no projeto, ambiente e decisões de otimização de desempenho.
2. **Stored Procedures:** Lógicas de negócio encapsuladas, parametrização e ganhos de segurança/desempenho.
3. **Functions e Views:** Uso de funções (escalares/tabela) para lógica reutilizável e views para simplificação de relatórios/consultas.
4. **Triggers:** Regras de integridade, controle de auditoria e justificativa técnica de impacto de desempenho.
5. **Transações e Controle de Concorrência:** Garantia das propriedades ACID nas operações críticas e gerenciamento de isolamento/locks.
6. **Cursors e FETCH:** Uso justificado (se aplicado) ou alternativas baseadas em conjuntos adotadas no projeto.
7. **Otimização de Consultas e Índices:** Análise do plano de execução, estrutura de índices criados e ferramentas de monitoramento utilizadas.
8. **Segurança e Boas Práticas:** Controle de acessos, prevenção a SQL Injection e organização/documentação do código T-SQL.

⚠ **Sessão Obrigatória na Documentação (Produção & Futuro):**

- **Colocação em Produção:** Cuidados essenciais, infraestrutura necessária e pontos de atenção/monitoramento constante.
- **Evolução do Projeto:** Expectativas, limitações atuais e cronograma para ampliações futuras.

5. Apresentação do Projeto (Apresentacao_Projeto.pdf)

- Apresentação comercial/técnica no formato PDF.
- **Mínimo de 10 slides** abordando: Contexto do problema, arquitetura técnica, demonstração visual das telas, destaques do banco de dados e plano de produção/evolução.

6. Repositório (pasta do projeto entregue) Os arquivos do projeto devem possuir a estrutura proposta, seguindo o formato de documentação utilizado no github, como:

- **README.md estruturado com:**
    - Descrição detalhada do projeto e seus objetivos.
    - Guia passo a passo de instalação e execução do projeto.
    - Como rodar o script PowerShell .ps1 de inicialização do banco.
    - Exemplos práticos de chamadas e testes da API REST.
    - Descrição da estrutura das pastas api e frontend.

Checklist de Entrega e Avaliação

Item	Descrição do Requisito	Status	Pontuação
1. Pacote de Entrega	Entregue em arquivo .zip único com estrutura de pastas correta?	[ ]	____ / 0.5
2. Automação DB (.ps1)	O script PowerShell cria o BD, tabelas e popula os dados de teste sem erros?	[ ]	____ / 1.5
3. Objetos SQL	Há uso correto de Procedures, Functions, Views, Triggers e Índices no SQL Server?	[ ]	____ / 1.5
4. API REST	A API conecta ao SQL Server e expõe endpoints de CRUD funcionais?	[ ]	____ / 1.5
5. Front-End	A interface gráfica consome a API e realiza as operações do CRUD de forma fluida?	[ ]	____ / 1.5
6. Documentação (PDF)	Documento em PDF aborda todos os 8 temas das aulas e as seções de Produção/Evolução?	[ ]	____ / 1.5
7. Apresentação (PDF)	A apresentação contém no mínimo 10 slides e resume bem a solução técnica e de negócio?	[ ]	____ / 1.0
8. GitHub & README.md	README claro, com instruções de execução, testes da API e organização das pastas?	[ ]	____ / 1.0
TOTAL			**____ / 10.0**